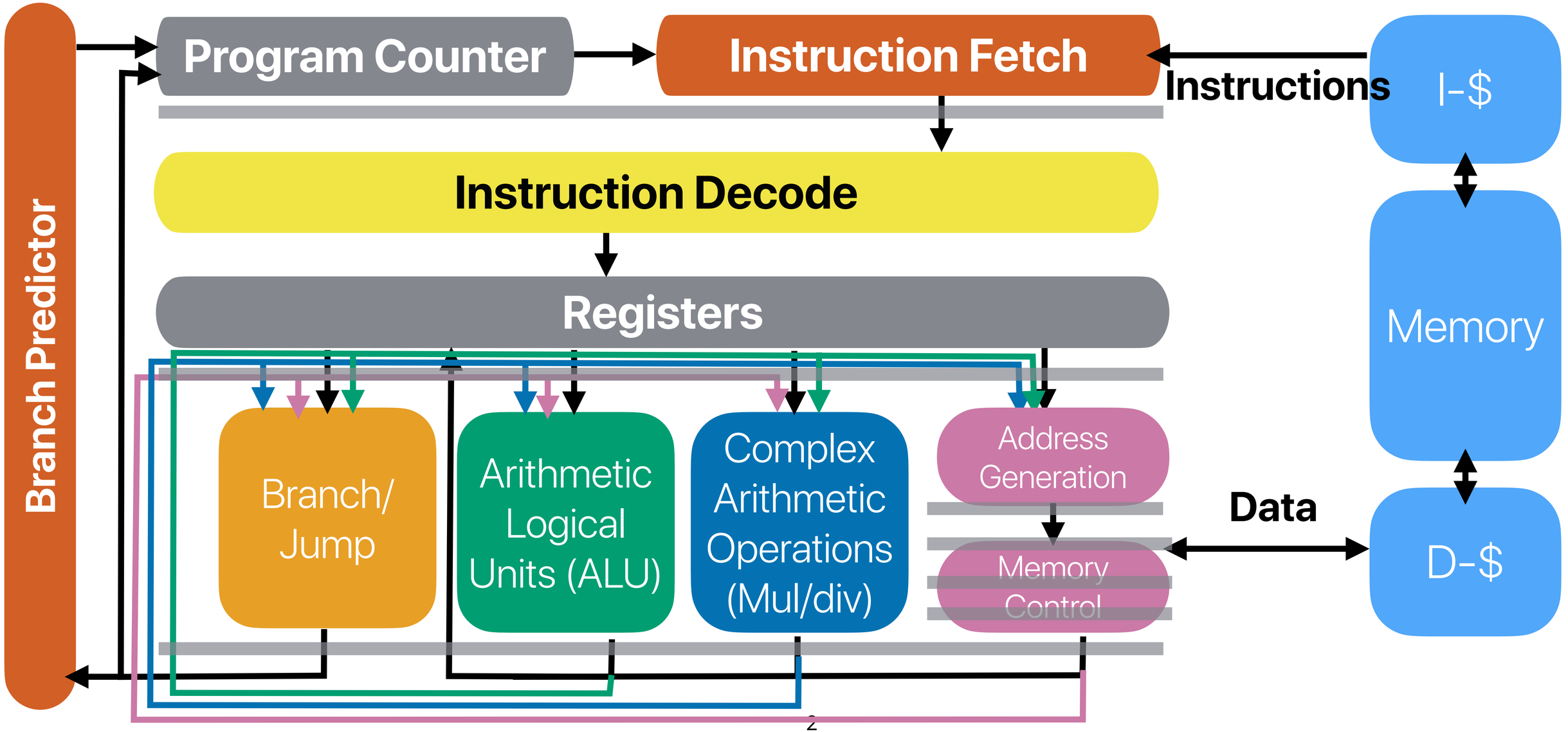
Modern Processor Design (IV): Try everything

Hung-Wei Tseng

Recap: Data "forwarding"



Takeaways: data hazards

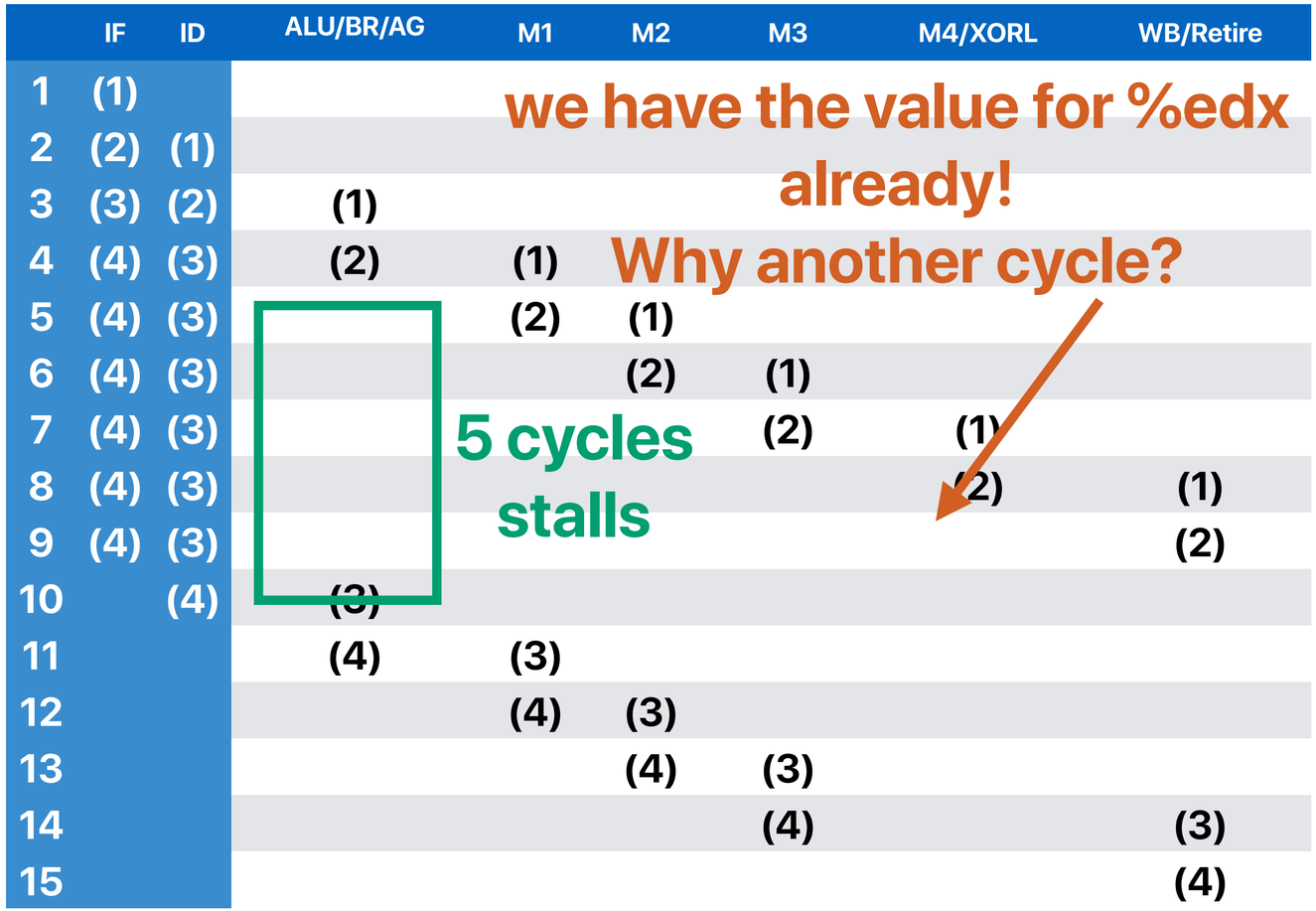
- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent

Data hazards?

- How many cycles do we have to stall in the following x86 instructions to get the expected output if a memory operation (assume 100% cache hit rate) takes 5 cycles?

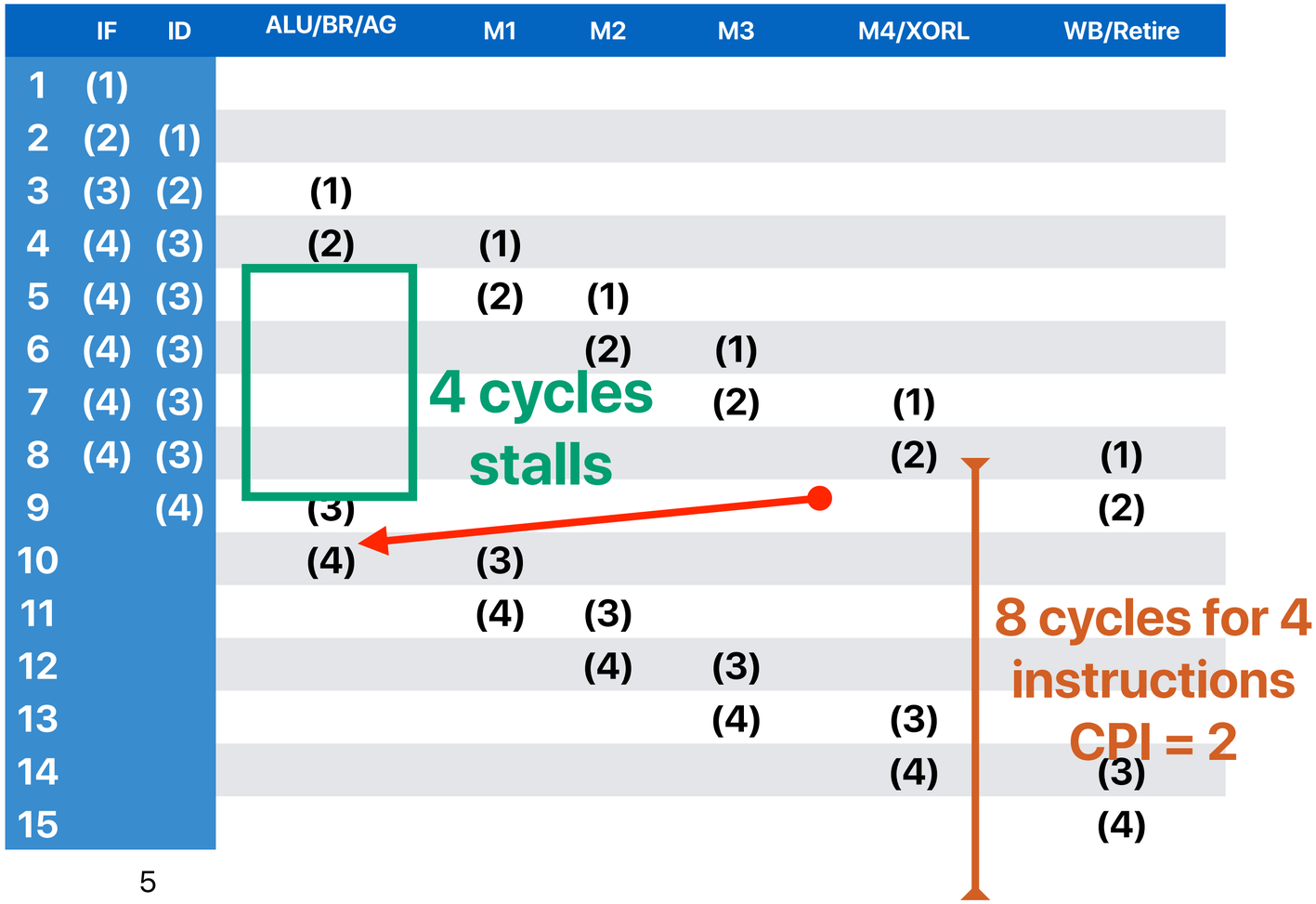
① `movl (%rdi), %eax`
② `movl (%rsi), %edx`
③ `movl %edx, (%rdi)`
④ `movl %eax, (%rsi)`

A. 1
B. 2
C. 3
D. 4
E. 5



The effect of data forwarding

```
① movl    (%rdi), %eax
② movl    (%rsi), %edx
③ movl    %edx, (%rdi)
④ movl    %eax, (%rsi)
```



More aggressive compiler optimization

```
for(i = 0; i < count; i++) {
    int64_t temp = a[i];
    a[i] = b[i];
    b[i] = temp;
}
```

```
movq    (%rdi,%rax), %rsi
movq    (%rcx,%rax), %r8
movq    %r8, (%rdi,%rax)
movq    %rsi, (%rcx,%rax)
addq    $8, %rax
cmpq    %r9, %rax
jne     .L9
movq    (%rdi,%rax), %rsi
movq    (%rcx,%rax), %r8
movq    %r8, (%rdi,%rax)
movq    %rsi, (%rcx,%rax)
addq    $8, %rax
cmpq    %r9, %rax
jne     .L9
```



```
.L9:
①  movq    (%rcx,%rax), %r8
②  movq    (%rdi,%rax), %rsi
③  addq    $8, %rax
④  movq    %r8, -8(%rdi,%rax)
⑤  movq    %rsi, -8(%rcx,%rax
⑥  cmpq    %r9, %rax
⑦  jne     .L9
⑧  movq    (%rcx,%rax), %r8
⑨  movq    (%rdi,%rax), %rsi
⑩  addq    $8, %rax
⑪  movq    %r8, -8(%rdi,%rax)
⑫  movq    %rsi, -8(%rcx,%rax
⑬  cmpq    %r9, %rax
⑭  jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(5)	(4)			(2)	(1)		
7	(5)	(4)				(2)	(1)	
8	(6)	(5)	(4)				(2)	(1)
9	(7)	(6)	(5)	(4)				(2)
10	(8)	(7)	(6)	(5)	(4)			(3)
11	(9)	(8)	(7)		(5)	(4)		
12	(10)	(9)	(8)			(5)	(4)	
13	(11)	(10)	(9)	(8)			(5)	(4)
14	(11)	(10)	(10)	(9)	(8)			(5)
15	(11)	(10)			(9)	(8)		(6)
16	(11)	(10)				(9)	(8)	(7)
17	(12)	(11)	(11)				(9)	(8)
18	(12)	(11)	(12)	(11)				(9)
19	(13)	(12)	(13)	(12)	(11)			(10)
20	(14)	(13)	(14)		(12)	(11)		
21		(14)				(12)	(11)	
22							(12)	(11)
23								(12)
24								(13)
25								(14)

9 cycles for 7 instructions

CPI = 1.29

Missing opportunities: if we know the loop always run even times?

```
for(i = 0; i < count; i++) {
    int64_t temp = a[i];
    a[i] = b[i];
    b[i] = temp;
}
```

Compiler can only do this when it's 100% for sure count is always an even number! — loop unrolling

Compilers are limited by the number of registers available to the software!

```
:
movq    (%rcx,%rax), %r8
movq    (%rdi,%rax), %rsi
addq    $8, %rax
movq    %r8, -8(%rdi,%rax)
movq    %rsi, -8(%rcx,%rax)
cmpq    %r9, %rax
jne     .L9
movq    (%rcx,%rax), %r8
movq    (%rdi,%rax), %rsi
addq    $8, %rax
movq    %r8, -8(%rdi,%rax)
movq    %rsi, -8(%rcx,%rax
cmpq    %r9, %rax
jne     .L9
```

```
.L9:
⑥  movq    (%rcx,%rax), %r8
⑦  movq    (%rdi,%rax), %rsi
⑧  cmpq    %r9, %rax
⑨  jne     .L9
⑩  addq    $8, %rax
⑪  movq    %r8, -8(%rdi,%rax)
⑫  movq    %rsi, -8(%rcx,%rax)
⑬  cmpq    %r9, %rax
⑭  jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(5)	(4)			(2)	(1)		
7	(6)	(5)				(2)	(1)	
8	(6)	(5)	(4)				(2)	(1)
9	(7)	(6)	(5)	(4)				(2)
10	(8)	(7)	(6)	(5)	(4)			(3)
11	(9)	(8)	(7)	(6)	(5)	(4)		
12	(8)	(7)	(8)	(7)	(6)	(5)	(4)	
13	(11)	(10)	(9)		(7)	(6)	(5)	(4)
14	(12)	(11)	(10)			(7)	(6)	(5)
15	(13)	(12)	(11)	(10)			(7)	(6)
16	(14)	(13)	(12)	(11)	(10)			(7)
17		(14)	(13)	(12)	(11)	(10)		(8)
18			(14)	(13)	(12)	(11)	(10)	(9)
19				(14)	(13)	(12)	(11)	(10)
20					(14)	(13)	(12)	(11)
21						(14)	(13)	(12)
22							(14)	(13)
23								(14)

7 cycles for 7 instructions

CPI = 1

Limitations of Compiler Optimizations

- If the hardware (e.g., pipeline changes), the same compiler optimization may not be that helpful
- The compiler can only optimize on static instructions, but cannot optimize dynamic instruction
  - Compiler cannot predict branches
  - Compiler does not know if cache has the data/instructions

Missing opportunities: if we know the loop always run even times?

```
for(i = 0; i < count; i++) {
    int64_t temp = a[i];
    a[i] = b[i];
    b[i] = temp;
}
```

Opportunities of hiding data hazards through out-of-order execution!

```
:
movq    (%rcx,%rax), %r8
movq    (%rdi,%rax), %rsi
addq    $8, %rax
movq    %r8, -8(%rdi,%rax)
movq    %rsi, -8(%rcx,%rax)
cmpq    %r9, %rax
jne     .L9
movq    (%rcx,%rax), %r8
movq    (%rdi,%rax), %rsi
addq    $8, %rax
movq    %r8, -8(%rdi,%rax)
movq    %rsi, -8(%rcx,%rax
cmpq    %r9, %rax
jne     .L9
```

```
.L9:
①  movq    (%rcx,%rax), %r8
②  movq    (%rdi,%rax), %rsi
③  addq    $8, %rax
④  movq    %r8, -8(%rdi,%rax)
⑤  movq    %rsi, -8(%rcx,%rax
⑥  movq    (%rcx,%rax), %r8
⑦  movq    (%rdi,%rax), %rsi
⑧  cmpq    %r9, %rax
⑨  jne     .L9
⑩  addq    $8, %rax
⑪  movq    %r8, -8(%rdi,%rax)
⑫  movq    %rsi, -8(%rcx,%rax
⑬  cmpq    %r9, %rax
⑭  jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(5)	(4)			(2)	(1)		
7	(5)	(4)				(2)	(1)	
8	(6)	(5)	(4)				(2)	(1)
9	(7)	(6)	(5)	(4)				(2)
10	(8)	(7)	(6)	(5)	(4)			(3)
11	(9)	(8)	(7)	(6)	(5)	(4)		
12	(10)	(9)	(8)	(7)	(6)	(5)	(4)	
13	(11)	(10)	(9)		(7)	(6)	(5)	(4)
14	(12)	(11)	(10)			(7)	(6)	(5)
15	(13)	(12)	(11)	(10)			(7)	(6)
16	(14)	(13)	(12)	(11)	(10)			(7)
17		(14)	(13)	(12)	(11)	(10)		(8)
18			(14)	(13)	(12)	(11)	(10)	(9)
19				(14)	(13)	(12)	(11)	(10)
20					(14)	(13)	(12)	(11)
21						(14)	(13)	(12)
22							(14)	(13)
23								(14)

7 cycles for 7 instructions

CPI = 1

Missing opportunities

```
for(i = 0; i < count; i++) {
    int64_t temp = a[i];
    a[i] = b[i];
    b[i] = temp;
}
```

Processor can predict what should happen and unroll the loop "dynamically"

```
:
movq    (%rcx,%rax), %r8
movq    (%rdi,%rax), %rsi
addq    $8, %rax
movq    %r8, -8(%rdi,%rax)
movq    %rsi, -8(%rcx,%rax)
cmpq    %r9, %rax
jne     .L9
movq    (%rcx,%rax), %r8
movq    (%rdi,%rax), %rsi
addq    $8, %rax
movq    %r8, -8(%rdi,%rax)
movq    %rsi, -8(%rcx,%rax
cmpq    %r9, %rax
jne     .L9
```

```
.L9:
① movq    (%rcx,%rax), %r8
② movq    (%rdi,%rax), %rsi
③ addq    $8, %rax
④ movq    %r8, -8(%rdi,%rax)
⑤ movq    %rsi, -8(%rcx,%rax
⑥ movq    (%rcx,%rax), %r8
⑦ movq    (%rdi,%rax), %rsi
⑧ cmpq    %r9, %rax
⑨ jne     .L9
⑩ addq    $8, %rax
⑪ movq    %r8, -8(%rdi,%rax)
⑫ movq    %rsi, -8(%rcx,%rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(6)	(5)	(4)	(3)	(2)	(1)		
7	(5)	(4)				(2)	(1)	
8	(6)	(5)	(4)				(2)	(1)
9	(7)	(6)	(5)	(4)				(2)
10	(8)	(7)	(6)	(5)	(4)			(3)
11	(9)	(8)	(7)	(6)	(5)	(4)		
12	(10)	(9)	(8)	(7)	(6)	(5)	(4)	
13	(11)	(10)	(9)		(7)	(6)	(5)	(4)
14	(12)	(11)	(10)			(7)	(6)	(5)
15	(13)	(12)	(11)	(10)			(7)	(6)
16	(14)	(13)	(12)	(11)	(10)			(7)
17		(14)	(13)	(12)	(11)	(10)		(8)
18			(14)	(13)	(12)	(11)	(10)	(9)
19				(14)	(13)	(12)	(11)	(10)
20					(14)	(13)	(12)	(11)
21						(14)	(13)	(12)
22							(14)	(13)
23								(14)

7 cycles for 7 instructions

CPI = 1

Dynamic instruction scheduling/ Out-of-order (OoO) execution

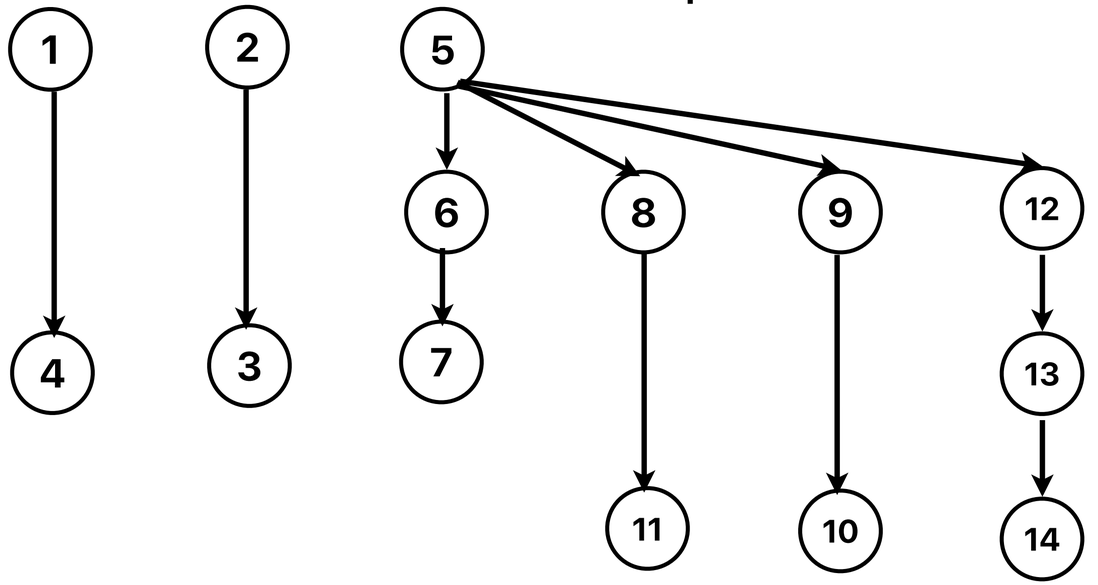
What do you need to execution an instruction?

- Whenever the instruction is decoded — put decoded instruction somewhere
- Whenever the inputs are ready — **all data dependencies are resolved**
- Whenever the target functional unit is available

Scheduling instructions: based on data dependencies

- Draw the data dependency graph, put an arrow if an instruction depends on the other.

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



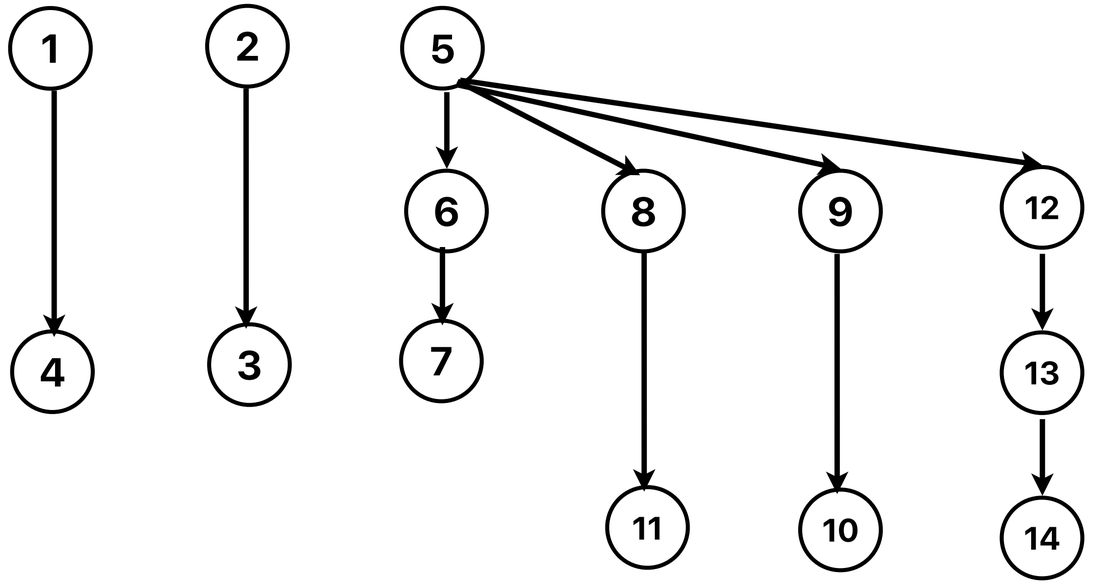
- **In theory**, instructions without dependencies can be executed in parallel or out-of-order
- Instructions with dependencies (on the same path) can never be reordered

https://www.pollev.com/hungweitseng close in 1:30

If we can predict the future …

- Consider the following dynamic instructions:

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



Which of the following pair can we reorder without affecting the correctness if the **branch prediction is perfect**?

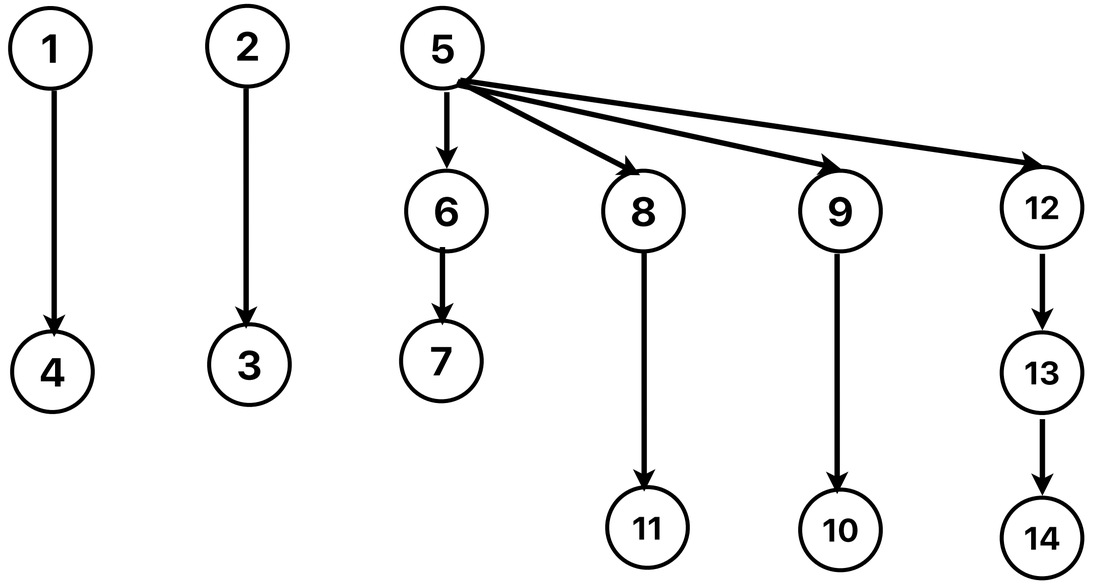
A. (1) and (2)
B. (2) and (5)
C. (4) and (5)
D. (3) and (8)
E. (6) and (12)



If we can predict the future …

- Consider the following dynamic instructions:

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



Which of the following pair can we reorder without affecting the correctness if the **branch prediction is perfect**?

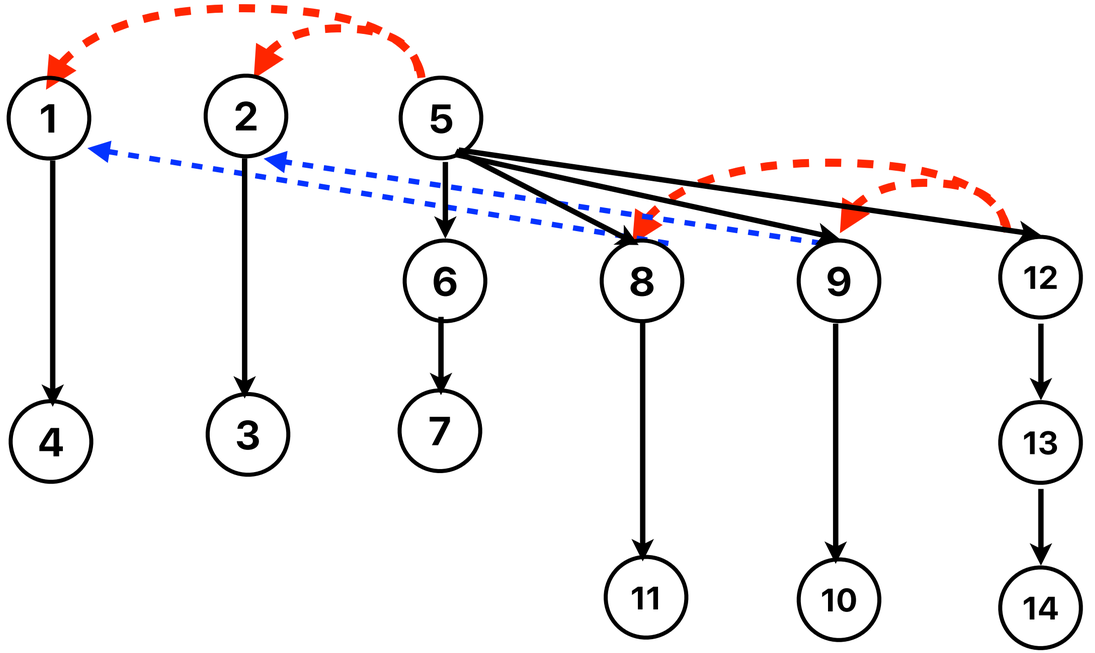
A. (1) and (2)
B. (2) and (5)
C. (4) and (5)
D. (3) and (8)
E. (6) and (12)

WAR (Write After Read): a later instruction (5) overwrites the source of an earlier one (2)
WAR (Write After Read): a later instruction (5) overwrites the source of an earlier one (2)
WAW (Write After Write): a later instruction (8) overwrites the output of an earlier one (4)
WAR (Write After Read): a later instruction (12) overwrites the source of an earlier one (6)

False dependencies

- We are still limited by **false dependencies**
- They are not "true" dependencies because they don't have an arrow in data dependency graph
  - WAR (Write After Read): a later instruction overwrites the source of an earlier one
  - WAW (Write After Write): a later instruction overwrites the output of an earlier one

```
①   movq    (%rdi,%rax), %rsi
②   movq    (%rcx,%rax), %r8
③   movq    %r8, (%rdi,%rax)
④   movq    %rsi, (%rcx,%rax)
⑤   addq    $8, %rax
⑥   cmpq    %r9, %rax
⑦   jne     .L9
⑧   movq    (%rdi,%rax), %rsi
⑨   movq    (%rcx,%rax), %r8
⑩   movq    %r8, (%rdi,%rax)
⑪   movq    %rsi, (%rcx,%rax)
⑫   addq    $8, %rax
⑬   cmpq    %r9, %rax
⑭   jne     .L9
```



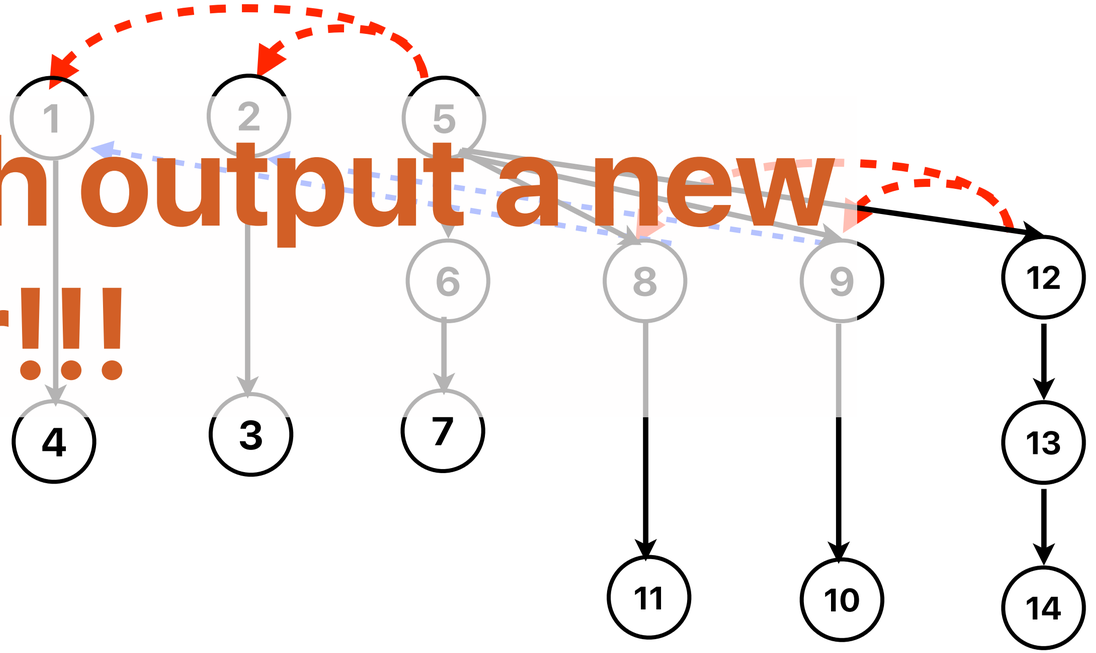
Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution

False dependencies

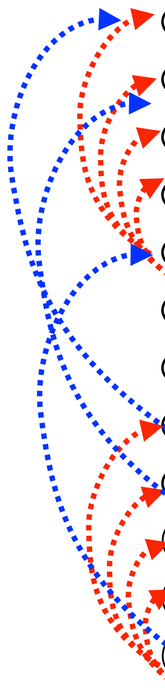
- We are still limited by **false dependencies**
- They are not "true" dependencies because they don't have an arrow in data dependency graph
  - WAR (Write After Read): a later instruction overwrites the source of an earlier one
  - WAW (Write After Write): a later instruction overwrites the output of an earlier one

```
①  movq   (%rdi,%rax), %rsi
②  movq   (%rcx,%rax), %r8
③  movq   %r8, (%rdi,%rax)
④  movq   %rsi, (%rcx,%rax)
⑤  addq   $8, %rax
⑥  cmpq   %r9, %rax
⑦  jne    .L9
⑧  movq   (%rdi,%rax), %rsi
⑨  movq   (%rcx,%rax), %r8
⑩  movq   %r8, (%rdi,%rax)
⑪  movq   %rsi, (%rcx,%rax)
⑫  addq   $8, %rax
⑬  cmpq   %r9, %rax
⑭  jne    .L9
```



What if we give each output a new register!!!

What if we can use more registers...



```
①  movq    (%rdi,%rax), %rsi
②  movq    (%rcx,%rax), %r8
③  movq    %r8, (%rdi,%rax)
④  movq    %rsi, (%rcx,%rax)
⑤  addq    $8, %rax
⑥  cmpq    %r9, %rax
⑦  jne     .L9
⑧  movq    (%rdi,%rax), %rsi
⑨  movq    (%rcx,%rax), %r8
⑩  movq    %r8, (%rdi,%rax)
⑪  movq    %rsi, (%rcx,%rax)
⑫  addq    $8, %rax
⑬  cmpq    %r9, %rax
⑭  jne     .L9
```



```
①  movq    (%rdi,%rax), %t0
②  movq    (%rcx,%rax), %t1
③  movq    %t1, (%rdi,%rax)
④  movq    %t0, (%rcx,%rax)
⑤  addq    $8, %rax, %t2
⑥  cmpq    %r9, %t2
⑦  jne     .L9
⑧  movq    (%rdi, %t2), %t3
⑨  movq    (%rcx, %t2), %t4
⑩  movq    %t4, (%rdi,%t2)
⑪  movq    %t3, (%rcx,%t2)
⑫  addq    $8, %t2, %t5
⑬  cmpq    %r9, %t5
⑭  jne     .L9
```

All false dependencies are gone!!!

The mechanism of OoO: Register renaming + speculative execution

- K. C. Yeager, “The MIPS R10000 superscalar microprocessor,” in IEEE Micro, vol. 16, no. 2, pp. 28-41, April 1996.

Register renaming + OoO

- Redirecting the output of an instruction instance to a **physical register**
- Redirecting inputs of an instruction instance from **architectural registers** to correct **physical registers**
  - You need a mapping table between architectural and physical registers
  - You may also need reference counters to reclaim physical registers
- OoO: Executing an instruction all operands are ready (the values of depending physical registers are generated)
  - You will need an **issue logic** to **issue** an instruction to the target functional unit

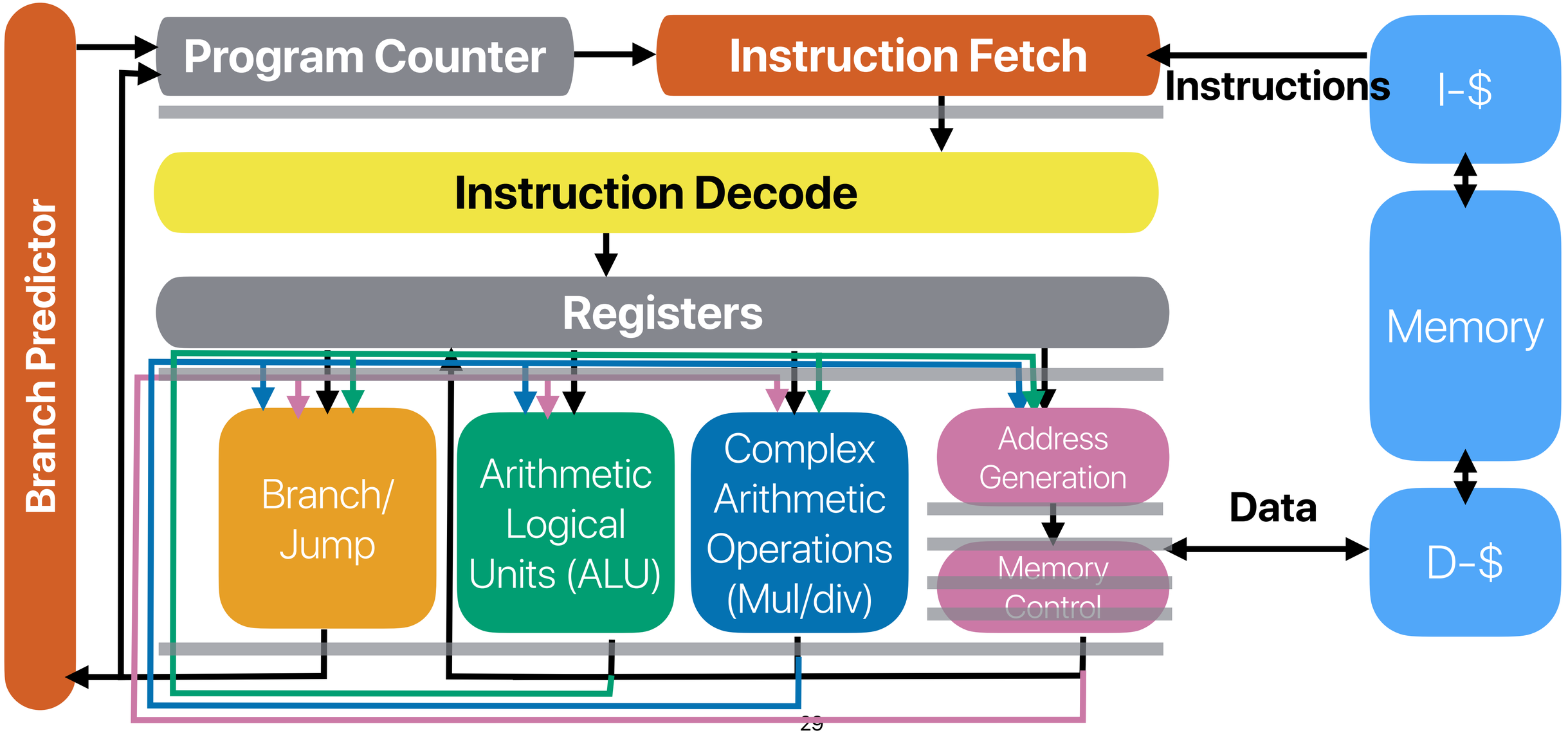
Can we really execute instructions OoO?

- Exceptions may occur anytime — divided by 0, page fault
  - A later instruction cannot write back its own result otherwise the architectural states won't be correct
  - Instructions after the one causes the exception should not be executed
- Hardware can schedule instruction across branch instructions with the help of branch prediction
  - Fetch instructions according to the branch prediction
  - However, branch predictor can never be perfect

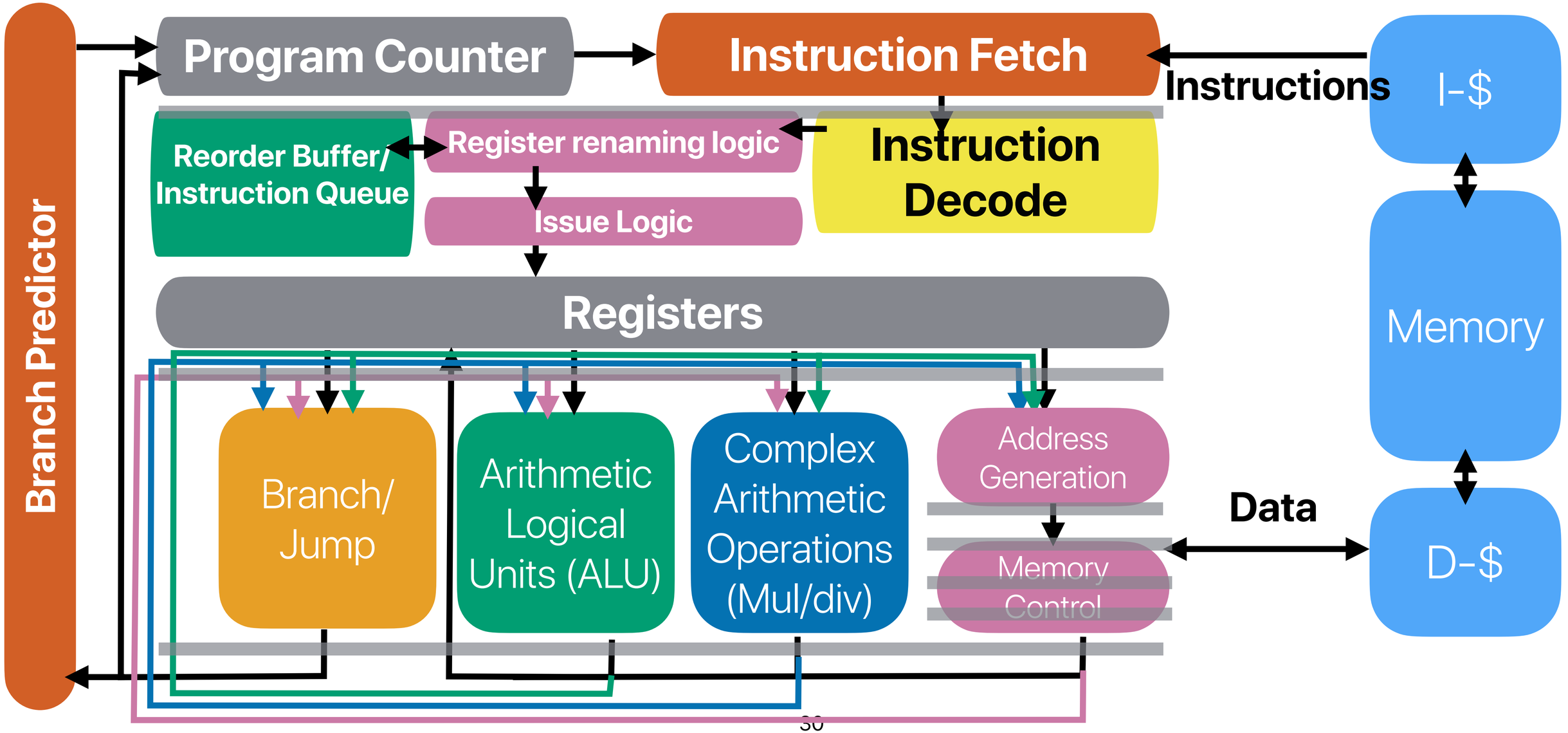
Speculative Execution

- **Speculative** execution mode: an executing instruction is considered as **speculative** before the processor hasn't determined if the instruction should be executed or not
- Reorder buffer (ROB)
  - The processor allocates an entry for each instruction in a reorder buffer
  - Store results in **reorder buffer and physical registers** when the instruction is still speculative
  - If an earlier instruction failed to commit due to an exception or mis-prediction, the physical registers and all ROB entries after the failed-to-commit instruction are flushed
- Commit/Retire
  - Present the execution result to the running program and in architectural registers when all prior instructions are non-speculative
  - Release the ROB entry

Data "forwarding"



Register renaming + OoO + RoB



Register renaming

```
①  movq (%rdi,%rax), %rsi
②  movq (%rcx,%rax), %r8
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4												
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	
rcx	
rdi	
rsi	
r8	

	Valid	Value	In use		Valid	Value	In use
P1				P6			
P2				P7			
P3				P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	
rcx	
rdi	
rsi	P1
r8	

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2				P7			
P3				P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8  (P1), (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3				P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3				P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8												
9												
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3				P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8												
9												
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3	0		1	P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9												
10												
11												
12												
13												
14												
15												
16												

Instruction (5) is running ahead of (3)

	Physical Register
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3	0		1	P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

Instruction (4) is running ahead of (3)

Instruction (5) is running ahead of (3)

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10												
11												
12												
13												
14												
15												
16												

	Physical Register
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	0		1	P7			
P3	1		1	P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10												
11												
12												
13												
14												
15												
16												

Retire/Commit (1)

	Physical Register
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	0		1	P7			
P3	1		1	P8			
P4				P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11												
12												
13												
14												
15												
16												

	Physical Register
rax	P3
rcx	
rdi	
rsi	P4
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5				P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12												
13												
14												
15												
16												

	Physical Register
rax	P3
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13												
14												
15												
16												

	Physical Register
rax	P3
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14												
15												
16												

	Physical Register
rax	P3
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax          → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15												
16												

	Physical Register
rax	P6
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6	0		1
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax          → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			(3)(4)(5)(6)(7)
16												

	Physical Register
rax	P6
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6	0		1
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax             → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax             → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)~~(7)
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)

	Physical Register
rax	P6
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6	1		1
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18												
19												
20												
21												
22												
23												

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax             → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax             → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				(8)(12)(13)(14)
19												
20												
21												
22												
23												

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							(9)(12)(13)(14)
20												
21												
22												
23												

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							~~(9)~~(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21												
22												
23												

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax             → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax             → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							~~(9)~~(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22												
23												

Register renaming

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							~~(9)~~(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22		(21)	(17)(18)(20)		(16)	(15)	(10)	(11)	(19)			(12)(13)(14)
23												

Register renaming

Only 1 of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax          → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							~~(9)~~(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22		(21)	(17)(18)(20)		(16)	(15)	(10)	(11)	(19)			(12)(13)(14)
23			(17)(20)(21)	(18)		(16)	(15)	(10)				(11)(12)(13)(14)(19)
24												
25												

Register renaming

Only 1 of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax          → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							~~(9)~~(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22		(21)	(17)(18)(20)		(16)	(15)	(10)	(11)	(19)			(12)(13)(14)
23			(17)(20)(21)	(18)		(16)	(15)	(10)				(11)(12)(13)(14)(19)
24			(20)(21)	(17)	(18)		(16)	(15)				(10)(11)(12)(13)(14)(19)
25												

Register renaming

Only 1 of them can have a instruction at the same cycle

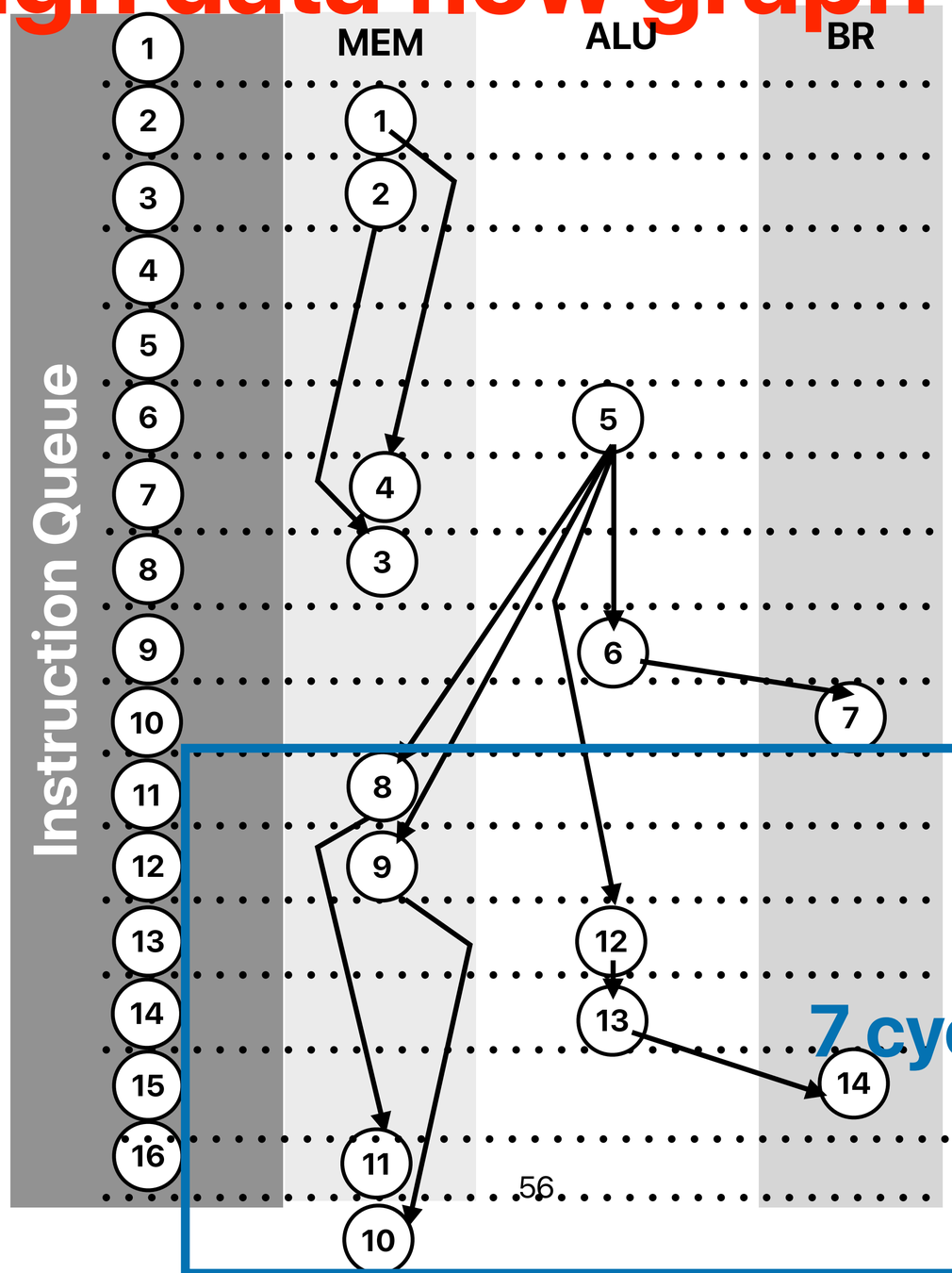
```
①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8 (P1), (%rdi,%rax)
④  movq %rsi(P2), (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax (P3)
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8 (P4), (%rdi,%rax)
⑪  movq %rsi(P11), (%rcx,%rax)
⑫  addq $8, %rax          → P6
⑬  cmpq %r9, %rax (P6)
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				~~(1)~~(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							~~(2)~~(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			~~(3)(4)(5)(6)(7)~~
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				~~(8)~~(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							~~(9)~~(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22		(21)	(17)(18)(20)		(16)	(15)	(10)	(11)	(19)			(12)(13)(14)
23			(17)(20)(21)	(18)		(16)	(15)	(10)				(11)(12)(13)(14)(19)
24			(20)(21)	(17)	(18)		(16)	(15)				~~(10)(11)(12)(13)(14)(19)~~
25			(21)		(17)	(18)		(16)	(20)			(15)(19)

7 cycles for 7 instructions
CPI = 1

Through data flow graph analysis

```
①  movq (%rdi,%rax), %rsi
②  movq (%rcx,%rax), %r8
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```



7 cycles every iteration

CPI = $\frac{7}{7} = 1$!

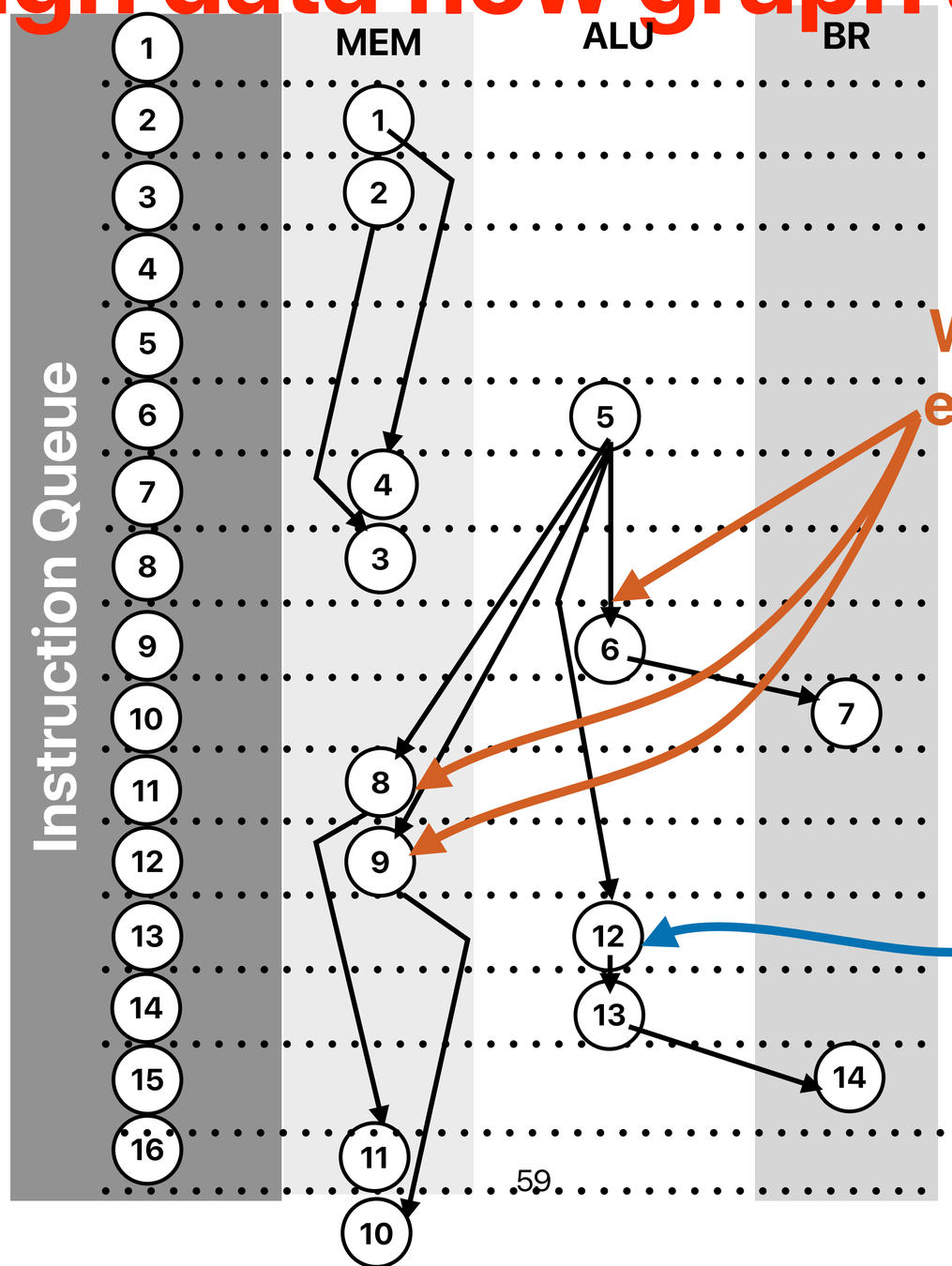
Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved

If CPI==1 the limitation?

Through data flow graph analysis

```
①  movq (%rdi,%rax), %rsi
②  movq (%rcx,%rax), %r8
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```



We cannot issue them earlier simply because structural hazards!

We could have this executed earlier if it's in the queue earlier

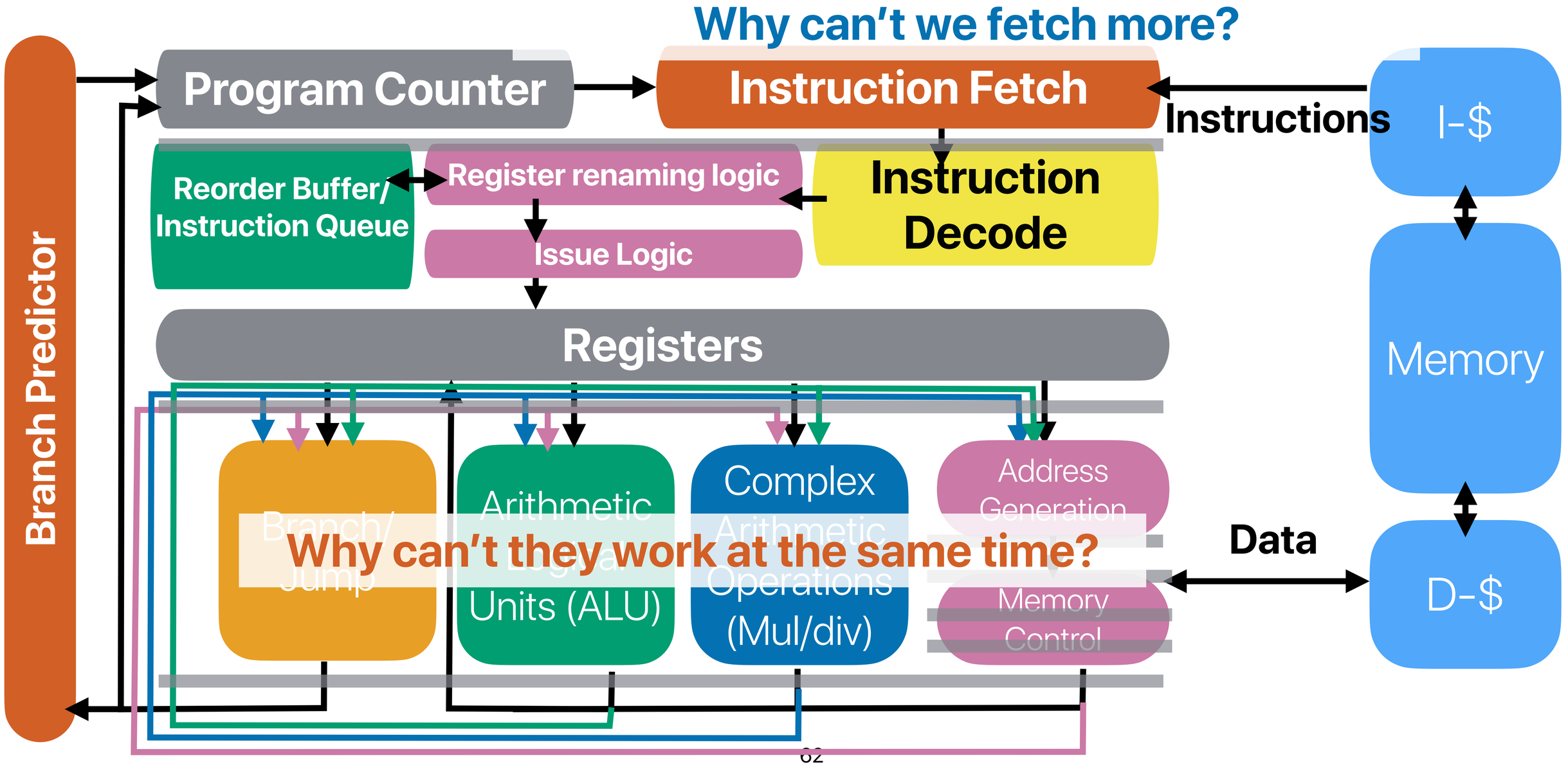
Super Scalar

Superscalar

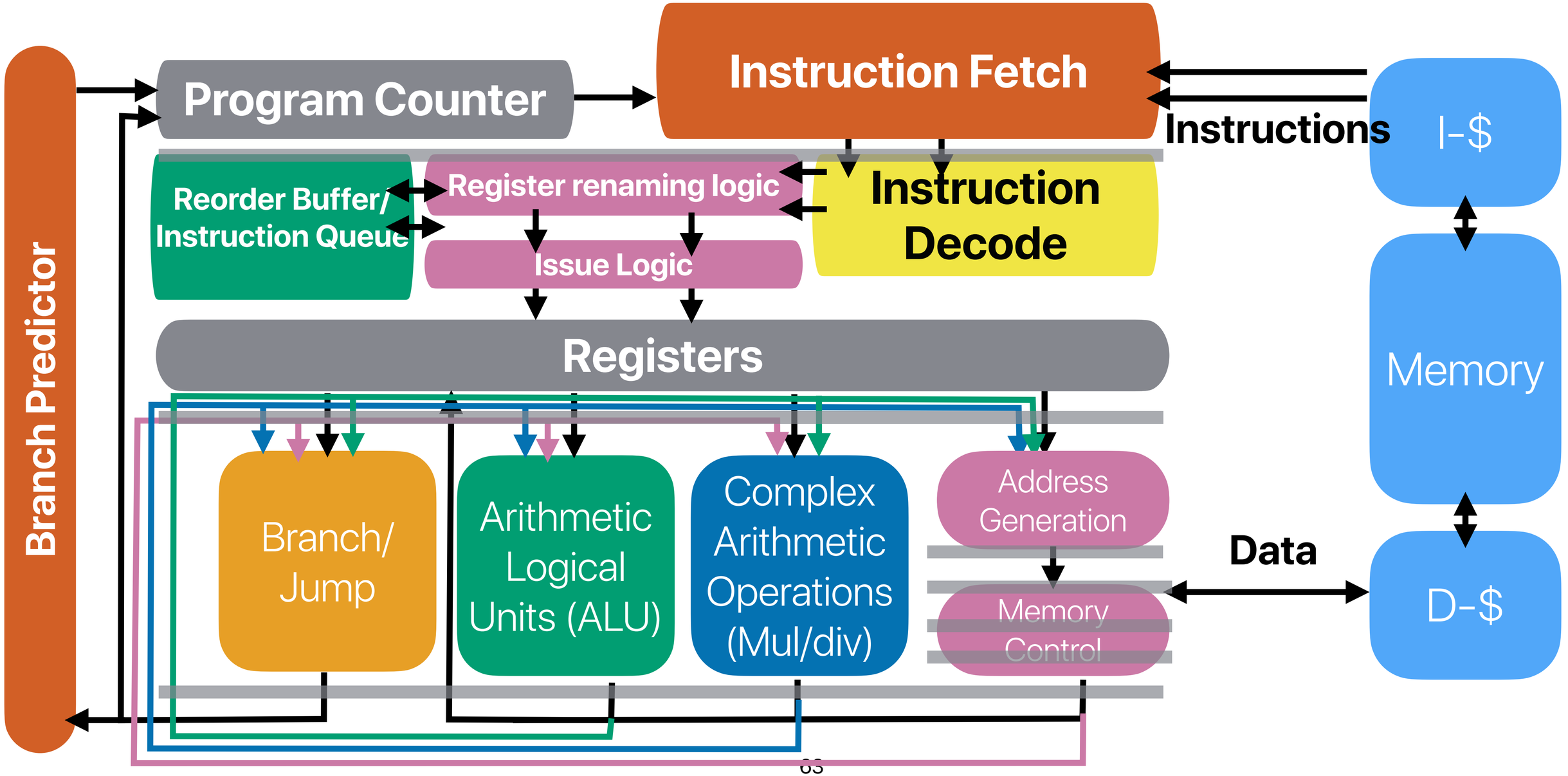
- Since we have many functional units now, we should fetch/decode more instructions each cycle so that we can have more instructions to issue!
- Super-scalar: fetch/decode/issue more than one instruction each cycle
  - **Fetch width:** how many instructions can the processor fetch/decode each cycle
  - **Issue width**: how many instructions can the processor issue each cycle
- The theoretical CPI should now be

$$\frac{1}{min(issue\ width, fetch\ width, decode\ width)}$$

Register renaming + OoO + RoB



Register renaming + SuperScalar



2-issue SS + Register renaming +OoO

2 issue: “2” of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4												
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: “2” of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)						
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)					(5)
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: “2” of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12												
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13												
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14												
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15												
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16												
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17												
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18												
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18						(17)		(16)				(10)(11)(12)(13)(14)(15)(19)(20)(21)
19												
20												

2-issue SS + Register renaming +OoO

2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			(1)(5)(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			(2)(5)(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	(3)(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18						(17)		(16)				(10)(11)(12)(13)(14)(15)(19)(20)(21)
19				(18)			(17)					(16)(19)(20)(21)
20												

2-issue SS + Register renaming +OoO

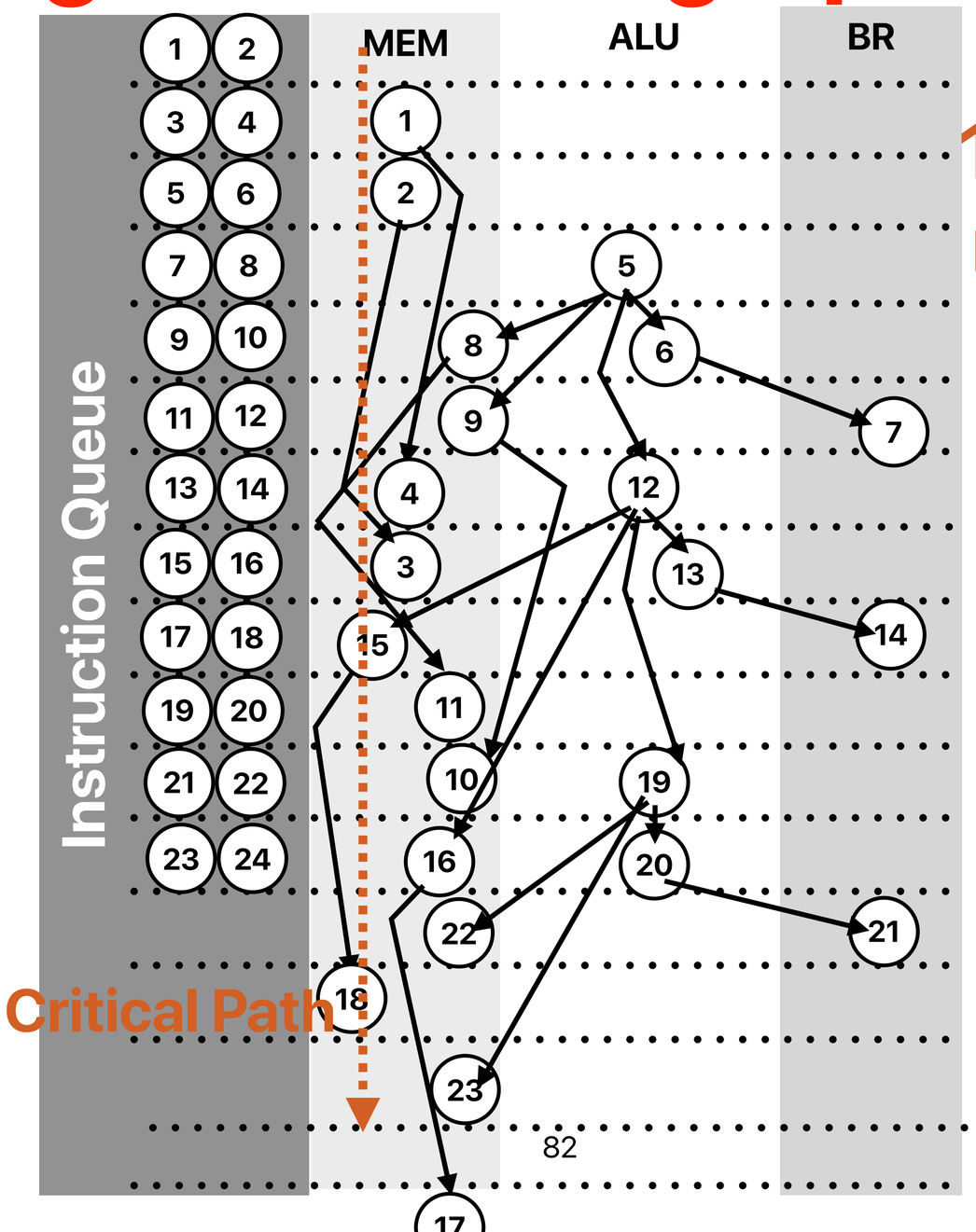
2 issue: "2" of them can have a instruction at the same cycle

```
①  movq (%rdi,%rax), %rsi   → P1
②  movq (%rcx,%rax), %r8    → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax            → P3
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi   → P4
⑨  movq (%rcx,%rax), %r8    → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax            → P6
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
```

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)(2)											
2	(3)(4)	(1)(2)										
3	(5)(6)	(3)(4)	(1)(2)									
4	(7)(8)	(5)(6)	(2)(3)(4)	(1)								
5	(9)(10)	(7)(8)	(3)(4)(5)(6)	(2)	(1)							
6	(11)(12)	(9)(10)	(3)(4)((6)(7)(8)		(2)	(1)			(5)			
7	(13)(14)	(11)(12)	(3)(4)((6)(7)(9)(10)	(8)		(2)	(1)		(6)			(5)
8	(15)(16)	(13)(14)	(3)(4)(7)(10)(11)(12)	(9)	(8)		(2)	(1)			(7)	(5)(6)
9	(17)(18)	(15)(16)	(3)(10)(11)(13)(14)	(4)	(9)	(8)		(2)	(12)			~~(1)(5)~~(6)(7)
10	(19)(20)	(17)(18)	(10)(11)(14)(15)(16)	(3)	(4)	(9)	(8)		(13)			~~(2)(5)~~(6)(7)(12)
11	(21)(22)	(19)(20)	(10)(11)(16)(17)(18)	(15)	(3)	(4)	(9)	(8)			(14)	(5)(6)(7)(12)(13)
12		(21)(22)	(16)(17)(18)(19)(20	(11)	(15)	(3)	(4)	(9)				(5)(6)(7)(8)(12)(13)(14)
13			(16)(17)(18)(20)(21)(22)	(10)	(11)	(15)	(3)	(4)	(19)			(5)(6)(7)(8)(9)(12)(13)(14)
14				(16)	(10)	(11)	(15)	(3)	(20)			(4)(5)(6)(7)(8)(9)(12)(13)(14)(19)
15					(16)	(10)	(11)	(15)			(21)	~~(3)(4)(5)(6)(7)(8)(9)~~(12)(13)(14)(19)(20)
16				(17)		(16)	(10)	(11)				(12)(13)(14)(15)(19)(20)(21)
17					(17)		(16)	(10)				(11)(12)(13)(14)(15)(19)(20)(21)
18						(17)		(16)				~~(10)(11)(12)(13)(14)(15)(19)~~(20)(21)
19				(18)			(17)					~~(16)~~(19)(20)(21)
20					(18)			(17)				(19)(20)(21)

Through data flow graph analysis

```
①  movq (%rdi,%rax), %rsi
②  movq (%rcx,%rax), %r8
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax
⑥  cmpq %r9, %rax
⑦  jne  .L9
⑧  movq (%rdi,%rax), %rsi
⑨  movq (%rcx,%rax), %r8
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax
⑬  cmpq %r9, %rax
⑭  jne  .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne  .L9
㉒  movq (%rdi,%rax), %rsi
㉓  movq (%rcx,%rax), %r8
㉔  movq %r8, (%rdi,%rax)
㉕  movq %rsi, (%rcx,%rax)
㉖  addq $8, %rax
㉗  cmpq %r9, %rax
```



12 cycles for every 11 memory instructions

If we have 11 loops, it will have 44 memory instructions, 77 instructions in total and take 48 cycles

CPI:

$$\frac{48}{77} = 0.62$$

Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved
- Super scalar further improves the utilization of hardware and throughput

The pipelines of Modern Processors

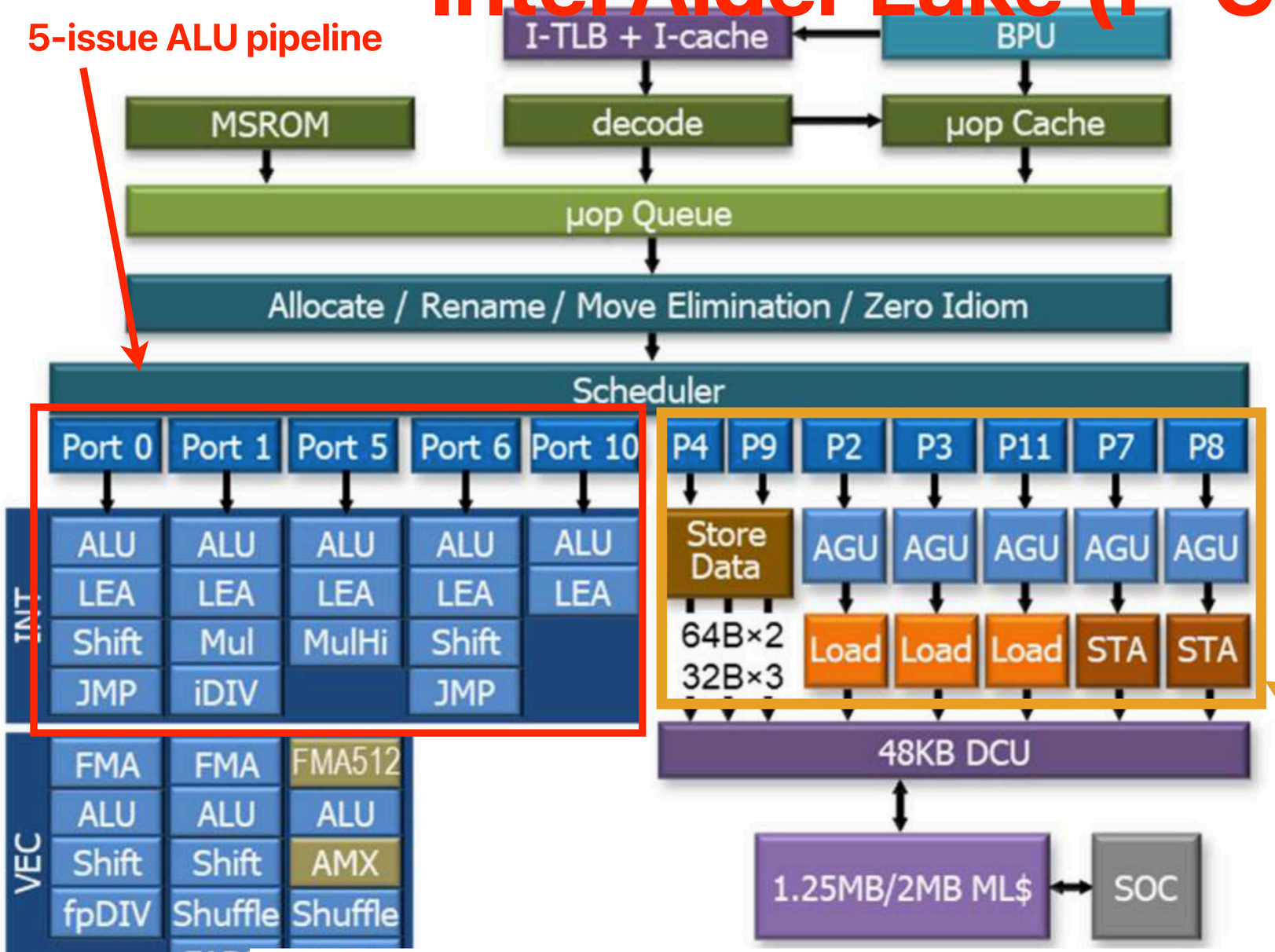
Intel Alder Lake (P-Core)

$$MinCPI = \frac{1}{12}$$

$$MinINTInst.CPI = \frac{1}{5}$$

$$MinMEMInst.CPI = \frac{1}{7}$$

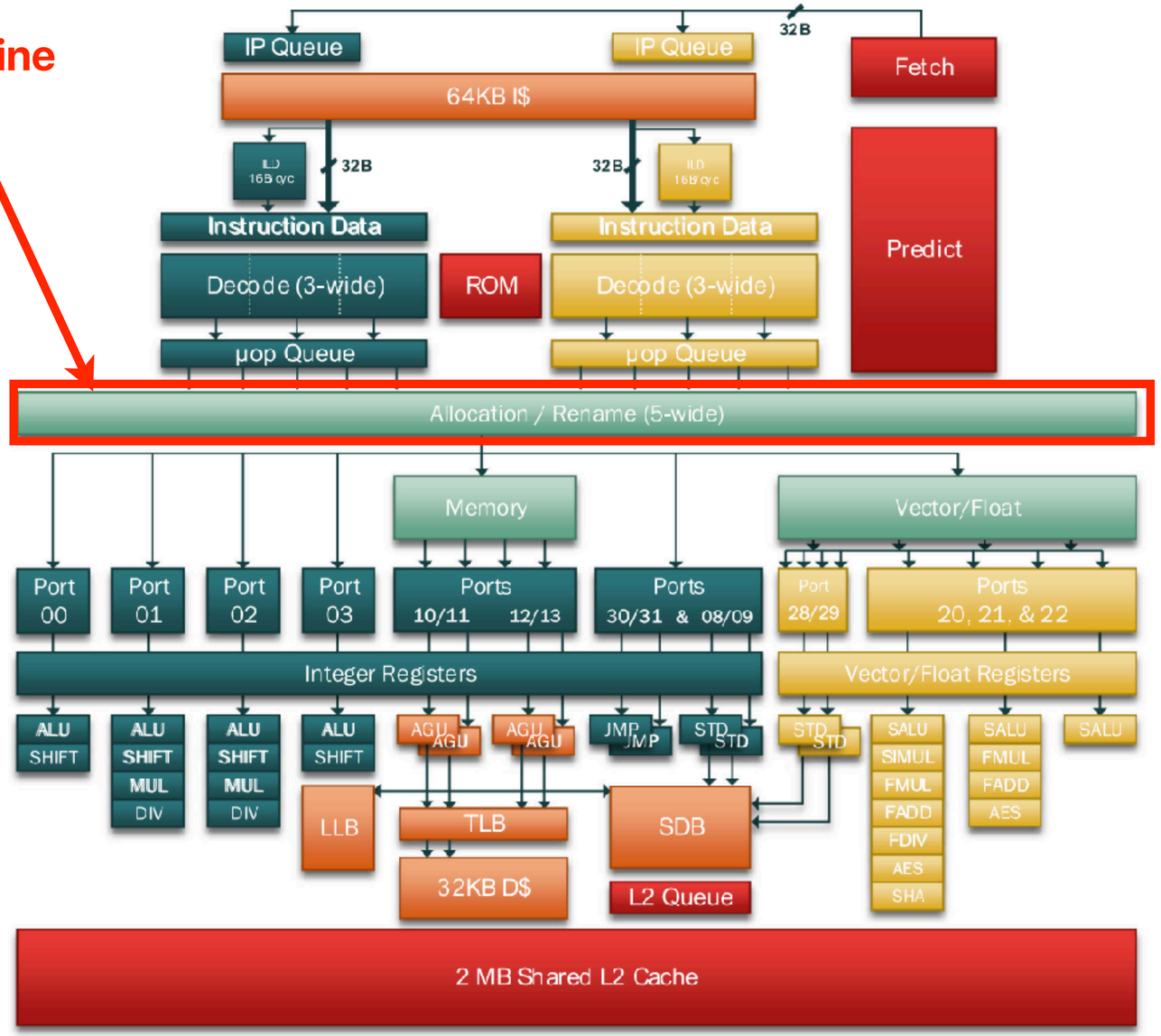
$$MinBRInst.CPI = \frac{1}{2}$$



https://download.intel.com/newsroom/2021/client-computing/intel-architecture-day-2021-presentation.pdf

Intel Alder Lake (E-Core)

5-issue pipeline



AMD Zen 3 (RyZen 5000 Series)

3-issue memory pipeline

4-issue integer pipeline + 1 additional branch

$$MinCPI = \frac{1}{8}$$

$$MinINTInst\,.\,CPI = \frac{1}{4}$$

$$MinMEMInst\,.\,CPI = \frac{1}{3}$$

$$MinBRInst\,.\,CPI = \frac{1}{2}$$

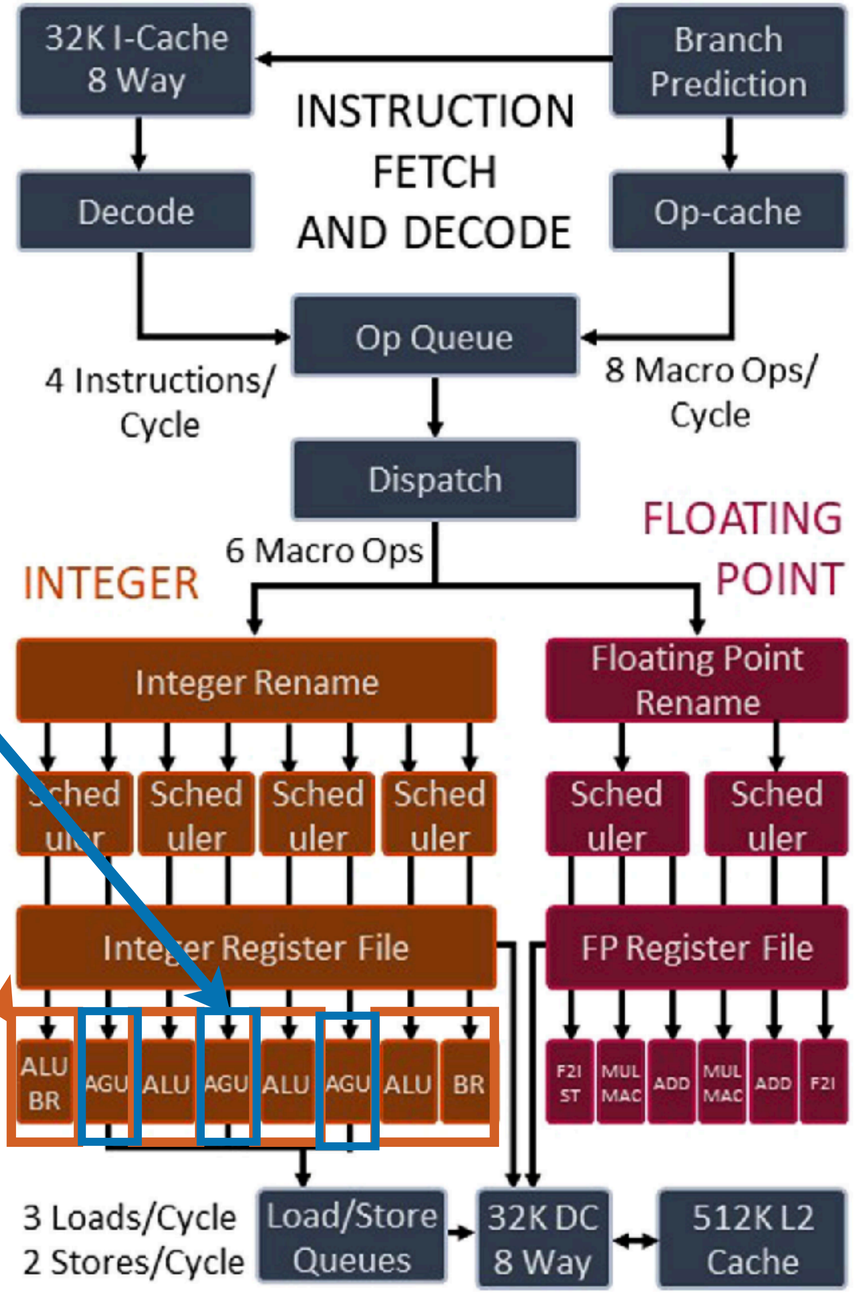


FIGURE 1. "Zen 3" block diagram.

Summary: Characteristics of modern processor architectures

- Multiple-issue pipelines with multiple functional units available
  - Multiple ALUs
  - Multiple Load/store units
  - Dynamic OoO scheduling to reorder instructions whenever possible
- Cache — very high hit rate **if your code has good locality**
  - Very matured data/instruction prefetcher
- Branch predictors — very high accuracy **if your code is predictable**
  - Perceptron
  - TAGE

Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved
- Super scalar further improves the utilization of hardware and throughput
- Modern processors are all very wide-issue super scalar processors with OoO capabilities

https://www.pollev.com/hungweitseng close in 1:30

Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and unlimited fetch/issue width, please identify the correct statements.
  - ① If the dataset is large, A will outperform B
  - ② If the dataset is small, there is very little performance difference between A and B
  - ③ If the dataset is small, B will outperform A as A has more branch instructions
  - ④ If the dataset is small, A will outperform B as A has fewer data dependencies

A. 0
B. 1
C. 2
D. 3
E. 4

A

```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

B

```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```



Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and unlimited fetch/issue width, please identify the correct statements.
  - ① If the dataset is large, A will outperform B
  - ② If the dataset is small, there is very little performance difference between A and B
  - ③ If the dataset is small, B will outperform A as A has more branch instructions
  - ④ If the dataset is small, A will outperform B as A has fewer data dependencies
  - A. 0
  - B. 1
  - C. 2
  - D. 3
  - E. 4

A

```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

B

```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```

Linked list v.s. arrays

- We can use either a linked list or an array to store a list of data, compare the performance of the array (version A) and linked list (version B) implementations that can achieve the same outcome as below. Assume we have a processor with a reasonably good branch predictor and unlimited fetch/issue width, please identify the correct statements.

① If the dataset is large, A will outperform B

② If the dataset is small, there is very little performance difference between A and B

③ If the dataset is small, B will outperform A as A has more branch instructions

④ If the dataset is small, A will outperform B as A has fewer data dependencies

A. 0

B. 1

C. 2

D. 3

E. 4

A

```
for(i=0;i<size;i++)
{
    if(node[i].next)
        number_of_nodes++;
}
```

B

```
while(node)
{
    node = node->next;
    number_of_nodes++;
}
```

Announcements

- **Assignment 4** due next **Thursday**
- **Reading Quiz 8** due **next Tuesday** before the lecture

Computer
Science &
Engineering

203

つづく

